Nodejs session 8

▼ Coding

```
// Index.js
require('dotenv').config({ path: './utils/.env' });
const cors = require('cors');
const  httpStatusText = require('./utils/httpStatusText');
const express = require('express');
const app = express();
const { title } = require('node:process');
const dns = require('dns');
dns.setServers(["8.8.8.8", "1.1.1.1"]);
const port = process.env.PORT ;

const mongoose = require('mongoose');

const url = process.env.MONGO_URL ;

mongoose.connect(url).then(() => {
    console.log("Connected to MongoDB");

});

app.use(cors());

app.use(express.json()); // Middleware to parse JSON reque
st bodies

const coursesRouter = require('./rout/courses_rouut');
const usersRouter = require('./rout/user_route');


app.use('/api/courses', coursesRouter);
app.use('/api/users', usersRouter);

// glopal  middleware for not found router
```

```
app.all(/.*/, (req, res) => {
    res.status(404).json({
        status: httpStatusText.ERROR,
        data: { message: 'Route not found' }
    });
});

// glopal  middleware for error handling
app.use((error, req, res, next) => {
    res.status(error.statusCode || 500).json({
        status: error.statusText || httpStatusText.ERROR,
        data: { message: error.message },
        code: error.statusCode || 500,
        datat: null
    });
});


app.listen(port, () => {
    console.log(`Server is running on http://localhost:${p
ort}`);
});

// User-Controle.js

const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const asyncWrapper = require('../middlewere/asyncWrappe
r');
const  httpStatusText = require('../utils/httpStatusTex
t');
const User = require('../models/user_model');
const AppError = require('../utils/appError');
const generateJWT = require('../utils/generateJWT');
require('dotenv').config({ path: '../utils/.env' });
```

```
const getAllUsers = asyncWrapper(async (req, res) => {
const query = req.query;
    const limit = query.limit || 10;
    const page = query.page || 1;
    const skip = (page - 1) * limit;
    //get all courses from the database
    const users = await User.find({},{"__v":false, "passwo
rd":false}).limit(limit).skip(skip);

    res.json({ status: httpStatusText.SUCCESS, data: { use
rs } });
})


const register = asyncWrapper(async (req, res, next) => {
    const { firstName, lastName, email, password } = req.b
ody;

    const oldUser = await User.findOne({ email: email });

    if(oldUser){
        const error = AppError.create('User already exist
s', 400, httpStatusText.FAILED);
        return next(error);
    }

    //password hashing can be added here for security
    const hashedPassword = await bcrypt.hash(password, 1
0);

    const user = new User
    ({
        firstName,
        lastName,
        email,
        password: hashedPassword
```

```
    });

    const token = await generateJWT({email: user.email, i
d: user._id});
    user.token = token;

    await user.save();

    res.status(201).json({ status: httpStatusText.SUCCESS,
data: { user } });
});

const login = asyncWrapper(async (req, res, next) => {
    // Implementation for user login
    const { email, password } = req.body;

    if (!email && !password) {
        const error = AppError.create('Email and password
are required', 400, httpStatusText.FAILED);
        return next(error);
    }

    const newMail = await User.findOne({ email: email });

    if (!newMail) {
        const error = AppError.create('User not found', 40
4, httpStatusText.FAILED);
        return next(error);
    }

    const matchedPassword = await bcrypt.compare(password,
newMail.password);

    if (newMail && matchedPassword) {
        const token = await generateJWT({email: newMail.em
```

```
ail, id: newMail._id});
        res.json({ status: httpStatusText.SUCCESS, data: {
token } });
    } else {
        const error = AppError.create('something wrong', 5
00, httpStatusText.ERROR);
        return next(error);
    }


});

module.exports = {
    getAllUsers,
    register,
    login
}

// VerifyToken.js
const jwt = require('jsonwebtoken');
require('dotenv').config({ path: '../utils/.env' });
const verifyToken = (req, res, next) => {
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1];

    const decoded = jwt.verify(token, process.env.JWT_SECR
ET_KEY);

    console.log(decoded);
    next();
}

module.exports = verifyToken;

//generateJWT.js
const jwt = require('jsonwebtoken');
```

```
require('dotenv').config({ path: './env' });

module.exports = async (pyload) => {
    //generate token for the user
    const token = await jwt.sign({
        email: user.email, id: user._id },
        process.env.JWT_SECRET_KEY,
        { expiresIn: '1m' }
    );
    return token;
}

// User-Route.js
const express = require('express');
const verifyToken = require('../middlewere/verfiyToken');

const router = express.Router();

const userController = require('../conrollers/users-contro
ler');

// Get all users


//register

//login


router.route('/')
    .get(verifyToken, userController.getAllUsers)


    router.route('/register')
    .post(userController.register)
```

```
    router.route('/login')
    .post(userController.login)



module.exports = router;

// userModel.js
const mongoose = require('mongoose');
const validator = require('validator');


const userSchema = new mongoose.Schema({
    firstName
    : {
        type: String,
        required: true
    },
    lastName: {
        type: String,
        required: true
    },
    email: {
        type: String,
        required: true,
        unique: true,
        validate: [ validator.isEmail, 'failed must be a v
alid email address' ]
    },
    password: {
        type: String,
        required: true,
        minlength: 6
    },
    token:{
        type: String,
```

```
    }
})




module.exports = mongoose.model('User', userSchema);
```

▼ This Is Explanation

👤 1) Users Model + Routes + Controller

📌 الفكرة

بنقسم الكود لثلاث أجزاء:

جزء	وظيفته
Model	شكل البيانات في DB
Controller	اللوجيك
Routes	تحديد الـ endpoints

🧠 ليه التقسيم ده؟

- تنظيم الكود
- scalable
- نفس طريقة الشركات

🧪 عملي

✅ Model (Mongoose)

```
constmongoose=require("mongoose");constuserSchema=newmongoose.Schema({
  name:String,
  email:String,
  password:String
});module.exports=mongoose.model("User",userSchema);
```

✅ Controller

```
exports.getUsers=async (req,res) => {constusers=awaitUser.find();res.json({ status:"su
ccess", data:users });
```

```
};
```

✅ Routes

```
constexpress=require("express");constrouter=express.Router();constuserController=requi
re("./controller");router.get("/",userController.getUsers);module.exports=router;
```

📝 2) Registration (إنشاء حساب)

📌 الفكرة

اليوزر يعمل signup → نخزن بياناته في DB

⚠️ أهم نقطة

❌ ممنوع تخزن password plain text

🧪 عملي

```
exports.register=async (req,res) => {const { name, email, password }=req.body;constuse
r=awaitUser.create({
    name,
    email,
    password
  });res.status(201).json({
    status:"success",
    data:user
  });
};
```

💡 ليه؟

- بداية نظام authentication
- كل حاجة بعد كده بتبني عليه

🔐 3) Password Hashing باستخدام bcrypt

📌 الفكرة

بدل ما نخزن password → نحوله لـ hash

🧠 ليه مهم؟

لو DB اتسربت 😈

→ الهاكر مش هيعرف الباسورد

🧪 عملي

تثبيت

```
npm install bcryptjs
```

hashing

```
constbcrypt=require("bcryptjs");consthashedPassword=awaitbcrypt.hash(password,10);
```

مقارنة (في login)

```
constisMatch=awaitbcrypt.compare(password,user.password);
```

🔥 الأفضل (في Model)

```
userSchema.pre("save",asyncfunction () {this.password=awaitbcrypt.hash(this.password,10);
```

```
});
```

💡 ليه ده أحسن؟

- أوتوماتيك
- يمنع نسيان hashing

🔑 4) Login

📌 الفكرة

اليوزر يدخل email + password → نتحقق

🧪 عملي

```
exports.login=async (req,res) => {const { email, password }=req.body;constuser=awaitUs
er.findOne({ email });if (!user) {returnres.status(404).json({
      status:"fail",
      message:"User not found"
    });
  }constisMatch=awaitbcrypt.compare(password,user.password);if (!isMatch) {returnres.s
tatus(401).json({
      status:"fail",
      message:"Incorrect password"
    });
  }res.json({
    status:"success",
    message:"Logged in successfully"
  });
};
```

💡 ليه؟

- التأكد من هوية المستخدم
- أساس الـ authentication

🪪 5) JWT (JSON Web Token)

📌 الفكرة

بدل session

→ بنستخدم token يتخزن عند اليوزر

🧠 بيحصل ايه؟

1. user يعمل login

2. السيرفر يدي token

3. اليوزر يبعت token في كل request

🧪 عملي

تثبيت

```
npm install jsonwebtoken
```

إنشاء token

```
constjwt=require("jsonwebtoken");consttoken=jwt.sign(
  { userId:user._id },"secretKey",
  { expiresIn:"1h" }
);
```

إرجاعه

```
res.json({
  status:"success",
  token
});
```

⚠️ مهم

- `.env` لازم يكون في secret

- متحطوش في الكود

🛡️ 6) Protected Routes

📌 الفكرة

في routes مش أي حد يدخلها

→ لازم يكون معاه token

🧪 عملي (middleware)

```
constjwt=require("jsonwebtoken");constprotect= (req,res,next) => {constauthHeader=req.
headers.authorization;if (!authHeader||!authHeader.startsWith("Bearer")) {returnres.st
atus(401).json({
      status:"fail",
      message:"Not authorized"
    });
  }consttoken=authHeader.split(" ")[1];try {constdecoded=jwt.verify(token,"secretKe
y");req.user=decoded;next();
  }catch (err) {res.status(401).json({
      status:"fail",
      message:"Invalid token"
    });
  }
};
```

استخدامه

```
app.get("/profile",protect, (req,res) => {res.json({
    status:"success",
    data:req.user
  });
});
```

💡 ليه؟

- حماية البيانات
- منع access غير مصرح

🔗 Flow كامل (من أول ما اليوزر يدخل)

1. 📝 Register
2. 🔐 Password يتعمله hash
3. 🔑 Login
4. 🎟️ JWT يتولد
5. 🛡️ Protected route يتحقق من التوكن

🧠 خلاصة مهمة جداً

الهدف	الحاجة
تخزين البيانات	User Model
إنشاء حساب	Registration
تأمين الباسورد	bcrypt
تحقق من المستخدم	Login
authentication بدون session	JWT
حماية endpoints	Protected Routes

💬 لو عايز المستوى اللي بعده 🔥

أقدر أشرحلك:

- Refresh Tokens
- Role-Based Access (admin/user)
- JWT attacks (algorithm confusion 😈)

- أو نبني Auth System كامل production-level

قولّي وندخل في التقيل 👨‍💻🔥